SQLD 제60회 기출 복원문제

각 문항 아래에 정답과 선택지별 풀이를 바로 확인할 수 있습니다.

정답·상세 풀이
총 50문항 · 1과목 10 / 2과목 40

1과목 데이터 모델링의 이해

01 1과목

발생 시점에 따라 엔터티를 분류할 때, 다른 엔터티로부터 업무 행위의 결과로 발생하며 데이터가 자주 변경되고 인스턴스의 양이 가장 많은 특징을 가지는 엔터티는 무엇인가?

① 기본 엔터티 ② 중심 엔터티 ③ 개념 엔터티 **④ 행위 엔터티**

정답 ④

핵심 해설

정답은 ④ 행위 엔터티다. 발생 시점 기준 엔터티는 기본(키)·중심·행위 엔터티로 나뉜다. 행위 엔터티는 두 개 이상의 다른 엔터티(주로 기본·중심 엔터티)로부터 발생하며, 업무가 진행될수록 계속 쌓이므로 데이터의 양이 많고 변경이 빈번하다(예: 주문, 이체, 접수).

선택지별 풀이

① 기본 엔터티는 다른 엔터티에 의존하지 않고 독립적으로 존재하며 자신의 주식별자를 가진다(예: 사원, 부서).

② 중심 엔터티는 기본 엔터티로부터 파생되어 업무의 중심을 이루는 엔터티다(예: 계약).

③ '개념 엔터티'는 발생 시점 분류의 표준 명칭이 아니라 오답 유인용 용어다.

> **보충 개념** 발생 시점 분류(기본/중심/행위)와 유무형 분류(유형/개념/사건)를 혼동하지 말 것.

02 1과목

다른 속성의 값을 계산하거나 특정 규칙으로 가공하여 저장하는 속성으로, 원본 속성이 바뀌면 값의 정합성 관리가 필요하고 되도록 최소화하는 것이 바람직한 속성은?

① 파생속성 ② 기본속성 ③ 설계속성 ④ 연관속성

정답 ①

핵심 해설

정답은 ① 파생속성이다. 파생속성(derived attribute)은 다른 속성이나 다른 엔터티의 속성값을 계산·가공하여 만들어진 속성이다(예: 나이=현재일-생년월일, 합계금액). 원천 데이터 변경 시 재계산이 필요해 정합성 관리 부담이 있으므로 꼭 필요한 경우에만 최소한으로 둔다.

선택지별 풀이

② 기본속성은 업무로부터 원래 도출되는 본래의 속성이다.

③ 설계속성은 업무상 존재하지 않지만 데이터 모델링·관리를 위해 새로 정의한 속성이다(예: 코드성 속성, 일련번호).

④ '연관속성'은 표준 분류 용어가 아닌 오답이다.

> **보충 개념** 속성은 특성에 따라 기본속성·설계속성·파생속성으로 분류된다.

03 1과목

엔터티, 인스턴스, 속성, 속성값 사이의 관계에 대한 설명으로 옳지 않은 것은?

① 하나의 엔터티는 두 개 이상의 인스턴스를 가진다.

② 하나의 엔터티는 두 개 이상의 속성을 가진다.

③ 하나의 인스턴스는 하나의 속성에 대해 두 개 이상의 속성값을 가진다.

④ 하나의 속성은 하나의 속성값을 가진다.

정답 ③

핵심 해설

정답(옳지 않은 것)은 ③이다. 하나의 인스턴스는 하나의 속성에 대해 오직 하나의 속성값만 가져야 한다(1NF, 원자값). 한 속성에 여러 값이 들어가면 다중값 속성으로 정규화(별도 엔터티 분리) 대상이 된다.

선택지별 풀이

① 엔터티는 두 개 이상의 인스턴스의 집합이므로 옳다.

② 엔터티는 두 개 이상의 속성을 가진다(속성이 하나뿐이면 엔터티로 성립하기 어렵다) — 옳다.

④ 하나의 인스턴스에서 하나의 속성은 하나의 속성값을 가진다 — 옳다.

> **보충 개념** 엔터티↔인스턴스, 속성↔속성값은 각각 '집합↔개체' 관계이며, '한 속성=한 값' 규칙이 관계형 모델의 원자성 전제다.

04 1과목

통신사 데이터 모델에 [고객], [서비스], [서비스이용], [청구], [납부] 엔터티가 있다. 한 고객은 여러 서비스를 이용할 수 있고 하나의 서비스는 여러 고객이 이용할 수 있다. 이 모델에 대한 설명으로 옳지 않은 것은?

① [서비스이용]은 [고객], [서비스] 어느 쪽과도 관계를 맺지 않는 독립 엔터티이다.

② [서비스이용]은 [고객]과 [서비스]의 M:N 관계를 해소하는 교차(관계) 엔터티이다.

③ [청구]는 [서비스이용]의 결과로 발생하는 행위 엔터티이다.

④ [납부]는 [청구]에 대응하여 발생한다.

정답 ①

핵심 해설

정답(옳지 않은 것)은 ①이다. [서비스이용]은 [고객]과 [서비스]의 다대다(M:N) 관계를 해소하기 위해 두 엔터티 사이에 도출된 교차(관계) 엔터티이므로, 양쪽 모두와 관계(외래식별자)를 맺는다. '어느 쪽과도 관계를 맺지 않는 독립 엔터티'라는 진술은 교차 엔터티의 본질과 정반대여서 틀렸다.

선택지별 풀이

② 고객·서비스가 M:N이면 반드시 교차 엔터티로 분해해야 하므로 옳다.

③ [청구]는 서비스 이용이라는 선행 사건으로부터 발생하는 행위 엔터티로 옳다.

④ [납부]는 [청구]에 대응(청구가 있어야 납부 발생)하므로 옳다.

> **보충 개념** M:N 관계는 관계형 모델에서 직접 구현되지 않아 교차 엔터티로 반드시 해소한다.

05 1과목

[고객]과 [주문] 엔터티가 "한 명의 고객은 0건 이상의 주문을 하고, 모든 주문은 반드시 한 명의 고객에 속한다"는 규칙으로 연결되어 있다. 이 관계에 대한 설명으로 옳은 것은?

① [고객] 쪽 참여는 필수(mandatory)이다.

② [주문] 쪽 참여는 선택(optional)이다.

③ 고객:주문의 관계 차수(cardinality)는 1:M이다.

④ 주문이 없는 고객은 존재할 수 없다.

정답 ③

핵심 해설

정답은 ③이다. "한 고객이 0건 이상의 주문"은 고객 1명에 주문이 여러 건(M) 대응할 수 있음을 뜻하므로 관계 차수는 1:M이다.

선택지별 풀이

① 고객은 주문이 "0건 이상"이므로 주문을 안 할 수도 있어 고객 쪽 참여는 선택(optional)이다 — 필수가 아니다.

② 모든 주문은 반드시 고객에 속하므로 주문 쪽 참여는 필수(mandatory)이다 — 선택이 아니다.

④ 고객은 주문이 0건일 수 있으므로 주문 없는 고객이 존재할 수 있다.

보충 개념 참여도(선택/필수)와 관계 차수(1:1·1:M·M:N)는 별개 개념이다. "0건 이상"은 선택적 참여, "반드시"는 필수적 참여를 뜻한다.

06 1과목

비식별자 관계(non-identifying relationship)에 대한 설명으로 옳지 않은 것은?

① 부모 엔터티의 주식별자가 자식 엔터티의 일반 속성(외래키)으로 상속된다.

② ERD에서 관계선을 점선으로 표현한다.

③ 자식 엔터티는 부모와 무관하게 자신만의 주식별자를 독립적으로 구성한다.

④ 부모 엔터티의 주식별자가 자식 엔터티의 주식별자를 그대로 구성한다.

정답 ④

핵심 해설

정답(옳지 않은 것)은 ④이다. '부모의 주식별자가 자식의 주식별자를 구성한다'는 것은 식별자 관계(identifying relationship)의 특징이다. 비식별자 관계에서는 부모의 주식별자가 자식으로 상속되되 자식의 '일반 속성(외래키)'이 되고, 자식은 자신의 독립된 주식별자를 갖는다.

선택지별 풀이

① 비식별자 관계에서 부모 키는 자식의 일반 속성으로 상속되므로 옳다.

② 비식별자 관계는 점선, 식별자 관계는 실선으로 표기하므로 옳다.

③ 자식이 독립적 주식별자를 가지는 것이 비식별자 관계의 핵심이므로 옳다.

보충 개념 식별자 관계는 강한 종속(부모 키가 자식 주식별자에 포함, 실선), 비식별자 관계는 약한 종속(일반 속성 상속, 점선)이다.

07 1과목

피터 첸(Peter Chen) 표기법에서 관계(Relationship)를 나타내는 데 사용하는 기호는?

① 사각형 ② 타원 **③ 마름모** ④ 직선(선)

정답 ③

핵심 해설

정답은 ③ 마름모다. 피터 첸 표기법에서는 엔터티=사각형, 속성=타원, 관계=마름모, 그리고 이들을 잇는 연결선으로 ERD를 표현한다.

선택지별 풀이

① 사각형은 엔터티를 나타낸다.

② 타원은 속성을 나타낸다.

④ 직선(선)은 엔터티·속성·관계를 연결하는 선일 뿐 관계 자체의 기호가 아니다.

보충 개념 실무에서 널리 쓰는 IE(까마귀발) 표기법이나 바커(Barker) 표기법에서는 관계를 선으로 표현해 피터 첸 표기법과 다르다. 표기법별 기호 차이를 구분해 둘 것.

08 1과목

주식별자를 구성하는 속성 중 하나라도 제외되면 각 인스턴스를 유일하게 구분할 수 없게 되어, 더 이상 줄일 수 없는 최소 속성 집합으로 구성해야 한다는 주식별자의 특성은?

① 대표성 **② 최소성** ③ 불변성 ④ 유일성

정답 ②

핵심 해설

정답은 ② 최소성이다. 최소성은 주식별자를 구성하는 속성이 유일성을 보장하는 데 꼭 필요한 최소한의 속성들로만 이뤄져야 한다는 특성이다. 구성 속성 중 하나만 빠져도 인스턴스 구분이 불가능해진다면 그 집합이 최소 집합임을 뜻한다.

선택지별 풀이

① 대표성은 주식별자가 엔터티를 대표하며 다른 엔터티와 참조 관계로 연결될 수 있어야 함을 의미한다.

③ 불변성은 주식별자 값이 자주 변하지 않아야 함을 의미한다.

④ 유일성은 주식별자로 모든 인스턴스를 유일하게 식별할 수 있어야 함을 의미한다 — 최소성이 전제하는 성질이지만 '하나라도 빠지면 안 되는 최소 집합'을 직접 규정하는 것은 최소성이다.

> **보충 개념** 주식별자의 4대 특성 = 유일성·최소성·불변성·존재성(대표성 포함하여 설명하기도 함).

09 1과목

정규화 단계 중, 기본키(주식별자)가 아닌 일반 속성들 사이의 함수적 종속을 제거하는 데 초점을 두어 주식별자와의 직접적 관련성이 가장 낮은 것은?

① 제1정규형 ② 제2정규형 **③ 제3정규형** ④ BCNF

정답 ③

핵심 해설

정답은 ③ 제3정규형이다. 제3정규형은 기본키가 아닌 일반 속성들 사이의 이행적 함수 종속을 제거하는 단계로, 초점이 "비(非)키 속성 간 종속"에 있어 주식별자와의 직접 관련성이 가장 낮다.

선택지별 풀이

① 제1정규형은 모든 속성을 원자값으로 만들고 기본키를 확정하는 단계로 키와 밀접하다.

② 제2정규형은 기본키(복합키)의 일부에만 종속되는 부분 함수 종속을 제거하므로 키와 직접 관련된다.

④ BCNF는 모든 결정자가 후보키가 되도록 하는 단계로 후보키(식별자)와 직접 관련된다.

> **보충 개념** 1NF·2NF·BCNF는 키를 기준으로 정의되는 반면, 3NF만 일반 속성 간 종속을 다룬다는 점이 핵심 구분이다.

10 1과목

인조 식별자(artificial identifier)에 대한 설명으로 옳은 것은?

① 업무적으로 자연스럽게 존재하여 사용자가 쉽게 인지할 수 있는 식별자이다.

② 원래 업무에 존재하던 속성을 그대로 주식별자로 사용한 것이다.

③ 업무적 의미와 무관하게 관리 편의를 위해 인위적으로 부여한 식별자이다.

④ 반드시 여러 속성을 조합하여 만든 복합 식별자만을 의미한다.

정답 ③

핵심 해설

정답은 ③이다. 인조 식별자는 본질 식별자로 삼을 마땅한 속성이 없거나 본질 식별자가 복잡할 때, 업무적 의미와 무관하게 일련번호 등으로 인위적으로 새로 부여한 식별자다(예: 주문번호 시퀀스). 특정 업무 행위나 의미가 없어도 부여할 수 있다.

선택지별 풀이

① 업무적으로 자연스럽게 존재하고 사용자가 쉽게 인지하는 것은 본질(본질/자연) 식별자의 특징이다.

② 원래 업무 속성을 그대로 쓰는 것도 본질 식별자에 해당한다.

④ 인조 식별자는 단일 속성(대개 일련번호)인 경우가 많고 복합 여부와 무관하므로 '복합만'이라는 진술은 틀렸다.

> **보충 개념** 대표성 유무 기준으로 주식별자/보조식별자를, 생성 여부 기준으로 본질/인조 식별자를 구분한다.

2과목 SQL 기본 및 활용

11 2과목

다음은 오라클 계층형 질의(CONNECT BY)에서 사용하는 의사컬럼·연산자에 대한 설명이다. 옳지 않은 것을 모두 고른 것은?

가. LEVEL은 계층의 깊이를 반환하며, 루트 노드는 0을 반환한다.
나. CONNECT_BY_ISLEAF는 해당 행이 루트 노드이면 1을 반환한다.
다. CONNECT_BY_ISCYCLE는 사이클이 발생하지 않은 정상 행에 대해 1을 반환한다.
라. CONNECT_BY_ROOT는 각 행에 대해 계층의 최하위(리프) 노드 값을 반환하는 연산자다.

① 나, 다 ② 나, 다, 라 ③ 가, 다, 라 **④ 가, 나, 다, 라**

정답 ④

핵심 해설
네 설명이 모두 틀렸으므로 정답은 ④(가·나·다·라)이다. 가: LEVEL은 루트 노드가 0이 아니라 1부터 시작한다(자식으로 내려갈수록 2,3,...). 따라서 오류. 나: CONNECT_BY_ISLEAF는 해당 행이 '리프(leaf)' 노드일 때 1을 반환한다. '루트'일 때가 아니므로 오류. 다: CONNECT_BY_ISCYCLE는 NOCYCLE 옵션과 함께 사용해 '사이클을 유발하는' 행에 1을 반환한다. '사이클이 없는 정상 행'이 아니므로 오류. 라: CONNECT_BY_ROOT는 각 행에 대해 계층의 '루트(최상위)' 노드 값을 반환하는 연산자다. '리프'가 아니므로 오류.

> **보충 개념** 계층형 질의 4대 요소로 LEVEL(깊이), CONNECT_BY_ISLEAF(리프 여부), CONNECT_BY_ISCYCLE(사이클 유발 여부, NOCYCLE 병행), CONNECT_BY_ROOT(루트 값)를 정확히 구분해 두어야 한다.

---

12 2과목

다음 SQL의 실행 결과로 옳은 것은? (단, 부서 테이블의 부서번호 컬럼에는 NULL이 포함된 행이 존재한다.)

```
SELECT 이름
FROM 사원
WHERE 부서번호 NOT IN (SELECT 부서번호 FROM 부서);
```

① 사원 테이블의 전체 행이 조회된다 ② 부서번호가 NULL인 사원만 조회된다
③ 한 건도 조회되지 않는다(공집합) ④ 문법 오류가 발생한다

정답 ③

핵심 해설
정답은 ③이다. NOT IN 서브쿼리 결과 집합에 NULL이 하나라도 포함되면, 비교 결과가 어떤 행에서도 TRUE가 될 수 없어 전체 결과가 공집합이 된다. 원리: x NOT IN (a, b, NULL)은 x<>a AND x<>b AND x<>NULL로 전개되는데, x<>NULL은 항상 UNKNOWN이다. AND 연산에서 하나라도 UNKNOWN이면 전체가 TRUE가 될 수 없으므로(최대 UNKNOWN) WHERE 절을 통과하는 행이 없다.

선택지별 풀이
① NULL이 없다면 미매칭 행이 나올 수 있지만, NULL이 있으므로 전체 조회는 불가.
② NOT IN은 NULL 행을 특별히 골라내지 않는다.
④ 문법 오류가 아니라 정상 실행되며 결과만 공집합이다.

> **보충 개념** 이 함정 때문에 실무·시험에서는 NOT IN 대신 NOT EXISTS를 권장한다. NOT EXISTS는 NULL에 영향받지 않는다.

---

13 2과목

부서별·직급별 연봉 합계를 다음과 같은 교차표 형태로 조회하려고 한다. 빈칸에 들어갈 올바른 구문은?

부서	사원	대리	팀장	부장
인사팀	...	...	...	...
IT팀	...	...	...	...
행정팀	...	...	...	...

```
SELECT *
FROM 급여       -- 컬럼: 부서, 직급, 연봉
( 빈칸 );
```

① `UNPIVOT (SUM(연봉) FOR 직급 IN ('사원','대리','팀장','부장'))`

② `PIVOT (직급 FOR SUM(연봉) IN ('사원','대리','팀장','부장'))`

③ `PIVOT (SUM(연봉) FOR 부서 IN ('인사팀','IT팀','행정팀'))`

④ `PIVOT (SUM(연봉) FOR 직급 IN ('사원','대리','팀장','부장'))`

정답 ④

핵심 해설

정답은 ④이다. PIVOT 구문은 PIVOT (집계함수(값컬럼) FOR 회전기준컬럼 IN (열로 펼칠 값 목록)) 형태다. 여기서 값은 연봉의 합계이므로 SUM(연봉), 열로 펼칠 기준은 직급이므로 FOR 직급 IN ('사원','대리','팀장','부장')가 되어야 한다. 남은 부서는 자동으로 행(그룹) 기준이 된다.

선택지별 풀이

① UNPIVOT은 반대 방향(열→행) 변환이므로 교차표를 만들지 못한다.

② 집계함수와 회전기준컬럼의 위치가 뒤바뀌어 문법 오류다.

③ 직급이 아니라 부서를 열로 펼치므로 요구한 결과(직급별 열)와 다르다.

> **보충 개념** PIVOT의 FOR 대상은 '열 헤더가 될 컬럼'이며, IN 목록의 값이 그대로 결과 열 이름이 된다.

---

14 2과목

급여가 각각 1000, 2000, 3000인 세 행이 있는 사원 테이블에 대해 다음 SQL을 실행했을 때, 결과 컬럼에 나타나는 값을 위에서부터 순서대로 나열한 것은?

```
SELECT 급여,
FIRST_VALUE(급여) OVER (ORDER BY 급여) AS 결과
FROM 사원;
```

① 1000, 1000, 1000

② 1000, 2000, 3000

③ 3000, 2000, 1000

④ 3000, 3000, 3000

정답 ①

핵심 해설

정답은 ①이다. FIRST_VALUE는 윈도우(정렬된 파티션) 내에서 '첫 번째' 값을 반환한다. ORDER BY 급여(오름차순)로 정렬하면 파티션의 첫 값은 최소값인 1000이며, 이 값이 모든 행에 대해 동일하게 반환된다. 따라서 결과는 1000, 1000, 1000이다. 주의: ORDER BY만 있고 윈도우 프레임을 명시하지 않으면 기본 프레임은 RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW다. FIRST_VALUE는 프레임의 시작(UNBOUNDED PRECEDING)을 가리키므로 어느 행에서 보든 첫 값 1000으로 고정된다.

선택지별 풀이

② 급여 원본값을 그대로 나열한 오답(윈도우 함수 미적용 착각).

③ 내림차순 정렬로 오해한 경우.

④ LAST_VALUE 기본 프레임 결과(현재 행까지의 마지막=자기 자신)와 혼동한 값.

> **보충 개념** 최소값 고정에 FIRST_VALUE+ORDER BY, 최대값에는 MAX() OVER() 또는 프레임을 확장한 LAST_VALUE를 쓴다.

---

15 2과목

다음 SQL의 WHERE 절과 논리적으로 동일한 조건은?

```
SELECT *
FROM 회원
WHERE (사번, 회원번호) IN ((10005, 2003), (10007, 2005));
```

① (사번=10005 OR 회원번호=2003) AND (사번=10007 OR 회원번호=2005)

② (사번=10005 AND 회원번호=2003) OR (사번=10007 AND 회원번호=2005)

③ 사번 IN (10005, 10007) OR 회원번호 IN (2003, 2005)

④ (사번=10005 AND 사번=10007) OR (회원번호=2003 AND 회원번호=2005)

정답 ②

핵심 해설

정답은 ②이다. 다중 컬럼 IN은 '튜플 단위' 비교다. 하나의 튜플 (10005, 2003)은 사번=10005 AND 회원번호=2003처럼 컬럼끼리 AND로 묶이고, IN 목록에 여러 튜플이 있으면 튜플들끼리는 OR로 연결된다. 따라서 (사번=10005 AND 회원번호=2003) OR (사번=10007 AND 회원번호=2005)가 정확히 동일하다.

선택지별 풀이

① 컬럼 내부를 OR로, 튜플 간을 AND로 뒤집어 의미가 완전히 달라진다.

③ 각 컬럼을 독립적으로 IN 비교하면 (10005, 2005)처럼 짝이 맞지 않는 조합까지 통과되어 결과가 넓어진다.

④ 같은 컬럼을 서로 다른 값과 AND로 묶어 항상 거짓이 되는 잘못된 전개다.

> **보충 개념** 다중 컬럼 IN의 핵심은 '컬럼=AND, 행=OR'이다.

16 2과목

8명의 사원을 급여 내림차순으로 정렬한 뒤 다음 SQL을 실행했다. 그룹 컬럼에 부여되는 값을 위에서부터 순서대로 나열한 것은?

```
SELECT 이름,
NTILE(3) OVER (ORDER BY 급여 DESC) AS 그룹
FROM 사원;
```

① 1,1,2,2,3,3,3,3

② 1,2,3,1,2,3,1,2

③ 1,1,1,2,2,2,3,3

④ 1,1,1,1,2,2,3,3

정답 ③

핵심 해설

정답은 ③(1,1,1,2,2,2,3,3)이다. NTILE(n)은 정렬된 행들을 n개 그룹으로 최대한 균등 분할한다. 8행을 3그룹으로 나누면 8 ÷ 3 = 2 나머지 2이므로, 앞쪽 그룹부터 나머지를 한 개씩 더 배분한다. 즉 그룹 크기는 3, 3, 2가 되어 1이 3개, 2가 3개, 3이 2개 → 1,1,1,2,2,2,3,3이다. ①·④: 그룹별 개수 배분(3,3,2)을 잘못 계산한 경우.
②: NTILE을 라운드로빈처럼 1,2,3을 번갈아 매기는 것으로 오해한 경우로, NTILE은 연속된 블록으로 나눈다.

> **보충 개념** 나머지가 있으면 '앞 그룹부터' 크기가 1씩 커진다. NTILE(4)로 8행이면 정확히 2,2,2,2.

17 2과목

두 테이블 부서와 사원을 결합하되, 매칭되지 않는 부서(사원이 한 명도 없는 부서)와 부서가 배정되지 않은 사원까지 양쪽 미매칭 행을 모두 결과에 포함시키려고 한다. 가장 적절한 방법은?

① NVL 함수를 사용한 CROSS JOIN

② NVL 함수를 사용한 INNER JOIN

③ FULL OUTER JOIN

④ UNION

정답 ③

핵심 해설

정답은 ③이다. FULL OUTER JOIN은 조인 조건이 일치하는 행은 물론, 왼쪽·오른쪽 테이블 각각에서 대응 행이 없는 행까지 모두 결과에 포함하고 부족한 쪽 컬럼은 NULL로 채운다. '양쪽 미매칭을 모두 표시'하는 요구에 정확히 부합한다.

선택지별 풀이

① CROSS JOIN은 두 테이블의 모든 조합(카티션 곱)을 만들 뿐 매칭/미매칭 개념이 없고, NVL로도 원하는 결합을 만들 수 없다.

② INNER JOIN은 양쪽이 일치하는 행만 남기므로 미매칭 행이 모두 사라진다. NVL을 붙여도 조인 결과 행 집합 자체는 늘지 않는다.

④ UNION은 컬럼 구조가 같은 두 결과 집합을 세로로 합칠 뿐, 두 테이블을 컬럼 기준으로 가로 결합하는 조인이 아니다.

> **보충 개념** FULL OUTER JOIN은 (LEFT OUTER JOIN) UNION (RIGHT OUTER JOIN)으로도 표현할 수 있다. Oracle 구식 표기 (+)로는 양방향 외부조인을 한 번에 표현할 수 없다.

18 2과목

사용자 R1이 자신의 스키마에 테이블 주문을 생성해 소유하고 있다. 사용자 R2가 SELECT * FROM R1.주문;을 실행하자 조회 권한이 없어 오류가 발생했다. R2가 해당 테이블을 조회할 수 있도록 하는 가장 올바른 방법은?

① R2가 CONNECT 롤을 다시 부여받는다

② R1이 GRANT SELECT ON 주문 TO R2;를 실행한다

③ R2가 자신의 스키마에 동일한 이름의 테이블을 새로 만든다

④ R2가 COMMIT을 실행한 뒤 다시 조회한다

정답 ②

핵심 해설

정답은 ②이다. 다른 사용자(R1)의 스키마 객체를 조회하려면 그 객체의 소유자가 SELECT 객체 권한을 부여해야 한다. 소유자 R1이 GRANT SELECT ON 주문 TO R2;를 실행하면 R2는 R1.주문을 조회할 수 있다.

선택지별 풀이

① CONNECT 롤은 DB에 접속할 수 있는 권한(시스템 권한 성격)일 뿐, 특정 테이블의 조회 권한을 주지 않는다. R2는 이미 접속에 성공했으므로 접속 권한 문제가 아니다.

③ 같은 이름의 빈 테이블을 자기 스키마에 만드는 것은 R1의 데이터를 조회하는 것과 무관하다.

④ COMMIT은 트랜잭션 확정이며 권한과 무관하다. 권한 부족 오류는 COMMIT으로 해소되지 않는다.

> **보충 개념** 권한 회수는 REVOKE SELECT ON 주문 FROM R2;. 여러 사용자에게 열려면 PUBLIC에 부여할 수도 있다.

19 2과목

다음 SQL의 실행 결과로 옳은 것은?

```
SELECT REGEXP_REPLACE('2026/02/25',
'([0-9]{4})/([0-9]{2})/([0-9]{2})',
'\3/\2/\1')
FROM DUAL;
```

① 2026/02/25 **② 25/02/2026** ③ 02/25/2026 ④ 2026/25/02

정답 ②

핵심 해설

정답은 ②(25/02/2026)이다. 패턴 안의 괄호는 캡처 그룹을 만든다. ([0-9]{4})가 그룹1='2026', 첫 ([0-9]{2})가 그룹2='02', 둘째 ([0-9]{2})가 그룹3='25'로 매칭된다. 치환 문자열 \3/\2/\1은 역참조로 그룹3/그룹2/그룹1을 순서대로 넣으므로 25/02/2026이 된다.

선택지별 풀이

① 원본 그대로로, 치환이 일어나지 않았다고 오해한 경우.

③ 그룹 순서를 3/2/1이 아니라 2/3/1(월/일/연) 등으로 잘못 대응한 경우.

④ 연도만 앞에 두고 뒤 두 그룹을 뒤집은 잘못된 대응.

보충 개념 역참조는 오라클에서 \1~\9로 표기한다(일부 환경은 백슬래시 이스케이프 주의). 날짜 형식 뒤집기(YYYY/MM/DD → DD/MM/YYYY)에 자주 쓰인다.

20 2과목

다음 실적 테이블에 대해 SELECT SUM(1분기), SUM(2분기) FROM 실적;을 실행했을 때의 결과로 옳은 것은?

사원	1분기	2분기
A	4	10
B	(NULL)	8
C	(NULL)	5

① **4, 23** ② NULL, 23 ③ 4, NULL ④ NULL, NULL

정답 ①

핵심 해설

정답은 ①(4, 23)이다. SUM을 포함한 집계함수는 대상 컬럼의 NULL을 '무시'하고 계산한다. 1분기는 4와 NULL 두 개인데 NULL을 제외하고 남은 4만 더해 4가 된다(0이나 NULL이 아니다). 2분기는 10+8+5 = 23이다.

선택지별 풀이

② NULL이 섞이면 합계가 NULL이 된다는 오해. 집계함수는 NULL을 건너뛰므로 유효값이 하나라도 있으면 결과는 NULL이 아니다.
③ 2분기가 NULL이라는 오해로, 2분기에는 NULL이 없다.
④ 두 컬럼 모두 NULL로 오해한 경우.

보충 개념 단, 대상 행이 전부 NULL이거나 조회 대상 행이 없으면 SUM 결과는 NULL이다(COUNT(*)만 0을 반환). NULL을 0으로 다루려면 SUM(NVL(1분기,0))을 쓴다. 또한 산술 연산(4 + NULL)은 NULL이 되지만, 집계함수의 NULL 무시와는 구분해야 한다.

21 2과목

Oracle에서 실행할 때, 다음 중 조회되는 행 수가 나머지 셋과 다른 것은? (사원 테이블에는 10건의 데이터가 있다.)

① `SELECT * FROM 사원 WHERE ROWNUM IN (1, 2);`

② `SELECT * FROM 사원 WHERE ROWNUM = 1;`

③ `SELECT * FROM 사원 WHERE ROWNUM <= 1;`

④ `SELECT * FROM 사원 WHERE ROWNUM < 2;`

정답 ①

핵심 해설

ROWNUM은 조건을 통과한 결과 행에 1부터 순차적으로 부여되는 의사(pseudo)컬럼이다.

선택지별 풀이

① **ROWNUM IN (1,2)는 첫 행에 1, 둘째 행에 2가 부여되어 두 행 모두 통과하므로 2건이다.**
② ROWNUM=1은 1건,
③ ROWNUM<=1은 1건,
④ ROWNUM<2 즉 ROWNUM=1도 1건이다. 따라서 ①만 2건으로 다르다. 참고로 ROWNUM은 1부터 부여되므로 ROWNUM=2나 ROWNUM>1처럼 첫 행이 통과하지 못하는 조건은 0건이 된다.

22 2과목

사원 테이블에 데이터가 있더라도 WHERE 1=2 조건으로 인해 대상 행이 하나도 없을 때, 다음 각 SQL의 실행 결과 중 나머지 셋과 다른 값을 반환하는 것은?

① `SELECT SUM(급여) FROM 사원 WHERE 1=2;`

② `SELECT AVG(급여) FROM 사원 WHERE 1=2;`

③ `SELECT MAX(급여) FROM 사원 WHERE 1=2;`

④ `SELECT COUNT(*) FROM 사원 WHERE 1=2;`

정답 ④

핵심 해설

집계함수는 대상 행이 하나도 없어도 항상 한 행을 반환한다. SUM·AVG·MAX·MIN은 빈 집합에 대해 NULL을 반환하지만, COUNT(*)는 행 개수를 세므로 0을 반환한다. 따라서 ①②③은 모두 NULL, ④만 0으로 값이 다르다. NVL(SUM(급여),0)처럼 감싸지 않는 한 SUM 등은 0이 아니라 NULL임에 유의한다.

23 2과목

부서 테이블의 모든 행을 보존하면서 사원 테이블과 조인하려고 한다. 다음 중 조인 결과가 나머지 셋과 다른 것은?

① `SELECT * FROM 부서 D LEFT OUTER JOIN 사원 E ON D.부서번호 = E.부서번호;`

② `SELECT * FROM 부서 D, 사원 E WHERE D.부서번호 = E.부서번호(+);`

③ `SELECT * FROM 부서 D, 사원 E WHERE D.부서번호(+) = E.부서번호;`

④ `SELECT * FROM 사원 E RIGHT OUTER JOIN 부서 D ON E.부서번호 = D.부서번호;`

정답 ③

핵심 해설

부서를 보존(왼쪽 보존)하려면 결핍이 생길 수 있는 사원 쪽에 (+)를 붙여야 한다.

선택지별 풀이

① ANSI LEFT OUTER JOIN(부서 보존),

② (+)가 사원측이므로 부서 보존,

④ 사원 RIGHT OUTER JOIN 부서는 부서 보존으로 셋 다 동일하다.

③은 (+)가 부서측에 붙어 사원을 보존하는 반대 방향 아우터 조인이 되므로 결과가 다르다. Oracle의 (+)는 NULL로 채워질(부족한) 쪽에 붙는다는 규칙을 기억한다.

24 2과목

다음 PIVOT 질의에 대한 설명으로 옳은 것은?

```
SELECT *
FROM (SELECT 부서, 직급 FROM 사원)
PIVOT (COUNT(*) FOR 직급 IN ('사원' AS 사원, '대리' AS 대리, '과장' AS 과장));
```

① 부서별로 행이 묶이고 직급 값이 열로 전개되어 각 직급 인원수가 표시된다.

② `PIVOT` 절에는 집계함수 없이 전개할 컬럼만 나열한다.

③ IN 절에 명시하지 않은 직급도 자동으로 결과 열에 나타난다.

④ 부서별 집계를 위해 반드시 GROUP BY 절을 함께 작성해야 한다.

정답 ①

핵심 해설

PIVOT은 행 값을 열로 전개하는 기능으로, 인라인 뷰에 나열되었으나 피벗 대상도 집계 대상도 아닌 컬럼인 부서가 자동으로 그룹 기준이 된다. FOR 직급 IN (...)에 나열된 값이 각각 열이 되고 COUNT(*)로 인원수가 채워진다.

선택지별 풀이

② PIVOT은 반드시 집계함수를 요구하고,
③ IN 절에 명시한 값만 열이 되며,
④ GROUP BY는 PIVOT이 암묵적으로 수행하므로 별도로 쓰지 않는다.

25 2과목

다음 SQL에서 서브쿼리가 두 건 이상을 반환하여 오류(ORA-01427)가 나는 일이 없도록 보장하려면, 부서 테이블의 부서명 컬럼에 어떤 제약조건이 필요한가?

```
SELECT 사원명
FROM 사원
WHERE 부서번호 = (SELECT 부서번호 FROM 부서 WHERE 부서명 = '영업');
```

① CHECK 제약조건
② UNIQUE 제약조건
③ NOT NULL 제약조건
④ FOREIGN KEY 제약조건

정답 ②

핵심 해설

= 연산자를 사용하는 단일행 서브쿼리이므로, 서브쿼리가 두 건 이상을 반환하면 'ORA-01427: 단일 행 하위 질의에서 2개 이상의 행이 리턴됨' 오류가 발생한다. 부서명에 UNIQUE 제약이 있으면 부서명='영업'을 만족하는 행이 최대 1건임이 보장되어 안전하다.
③ NOT NULL은 값의 존재만 보장할 뿐 행 개수를 제한하지 못하고, ①CHECK·④FK는 유일성과 무관하다.

26 2과목

다음 SQL이 급여 오름차순으로 첫 행부터 현재 행까지의 급여 누적합계를 계산하도록 (㉠)에 들어갈 윈도우 프레임 절로 옳은 것은?

```
SELECT 급여, SUM(급여) OVER (ORDER BY 급여 (㉠)) AS 누적합계
FROM 사원;
```

① `RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`
② `ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING`
③ `ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING`
④ `ROWS UNBOUNDED PRECEDING`

정답 ④

핵심 해설

ROWS UNBOUNDED PRECEDING은 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW와 같으며, 첫 행부터 현재 행까지를 누적하므로 누적합계가 된다.

선택지별 풀이

① 전체 행을 대상으로 해 모든 행이 총합으로 동일해지고,
② 앞뒤 1행씩(3행) 이동합계,
③ 현재 행부터 마지막까지의 역방향 누적이 된다.

27 2과목

트랜잭션의 특성(ACID) 중 고립성(Isolation)에 대한 설명으로 옳은 것은?

① 실행 중인 트랜잭션은 다른 트랜잭션의 연산에 영향을 받지 않는다.

② 트랜잭션의 연산은 전부 반영되거나 전혀 반영되지 않는다.

③ 트랜잭션이 성공적으로 완료되면 그 결과는 영구적으로 반영된다.

④ 트랜잭션 수행 전후에 데이터베이스는 일관된 상태를 유지한다.

정답 ①

핵심 해설

고립성(Isolation)은 동시에 수행되는 여러 트랜잭션이 서로의 중간 연산에 간섭받지 않고 독립적으로 수행되는 것처럼 보이는 성질이다.
②는 원자성(Atomicity, 전부 아니면 전무), ③은 지속성(Durability, 완료 결과의 영구 반영), ④는 일관성(Consistency, 무결성 유지)에 대한 설명이다.

28 2과목

기본키(Primary Key) 제약조건이 보장하는 특성으로 옳은 것은?

① NULL 값을 허용하며 컬럼 값은 유일(UNIQUE)해야 한다.

② NULL 값을 허용하며 값의 중복도 허용된다.

③ NULL 값을 허용하지 않으며(NOT NULL) 값은 유일(UNIQUE)해야 한다.

④ NULL 값을 허용하지 않으나(NOT NULL) 값의 중복은 허용된다.

정답 ③

핵심 해설

기본키는 행을 유일하게 식별하므로 NOT NULL(존재성)과 UNIQUE(유일성)를 동시에 만족해야 한다. 즉 NULL을 가질 수 없고 값이 중복될 수도 없다. ①②는 NULL을 허용한다는 점에서, ④는 중복을 허용한다는 점에서 틀리다. 참고로 UNIQUE 제약만 있는 컬럼은 NULL을 허용할 수 있다는 점이 기본키와의 차이다.

29 2과목

사원 테이블은 사원번호와 관리자번호(직속 상사의 사원번호)를 가진다. 최상위 관리자부터 하위 사원 방향으로 조직 계층을 전개하려 할 때, 올바른 계층 질의 절은? (START WITH 관리자번호 IS NULL을 사용한다고 가정)

① `CONNECT BY 사원번호 = PRIOR 관리자번호`

② `CONNECT BY PRIOR 사원번호 = 관리자번호`

③ `CONNECT BY PRIOR 관리자번호 = 사원번호`

④ `CONNECT BY 관리자번호 = 사원번호`

정답 ②

핵심 해설

하향(순방향) 전개에서는 PRIOR가 부모(상위) 행의 컬럼에 붙는다. 부모의 사원번호(PRIOR 사원번호)가 자식의 관리자번호와 같은 행을 연결하므로 CONNECT BY PRIOR 사원번호 = 관리자번호가 옳다. ①③은 PRIOR 위치가 반대라 상향(역방향) 전개가 되고, ④는 PRIOR가 없어 부모-자식 관계가 지정되지 않으므로 계층이 전개되지 않는다.

30 2과목

사원 테이블(사원번호, 관리자번호, 직급)에 대한 다음 계층 질의 중, 계층(트리)이 정상적으로 전개되지 않는 것은?

① `SELECT * FROM 사원 START WITH 관리자번호 IS NULL CONNECT BY PRIOR 사원번호 = 관리자번호;`

② `SELECT * FROM 사원 START WITH 사원번호 = 1000 CONNECT BY PRIOR 사원번호 = 관리자번호;`

③ `SELECT * FROM 사원 START WITH 관리자번호 IS NULL CONNECT BY PRIOR 사원번호 = 관리자번호 AND 직급 <> '인턴';`

④ `SELECT * FROM 사원 START WITH 관리자번호 IS NULL CONNECT BY 사원번호 = 관리자번호;`

정답 ④

핵심 해설

계층 질의에서 부모-자식 연결은 CONNECT BY 절의 PRIOR로 지정한다.
④는 CONNECT BY에 PRIOR가 없어 같은 행 내에서 사원번호=관리자번호를 비교할 뿐 상하위 관계를 전개하지 못하므로 루트 이후로 트리가 만들어지지 않는다. ①②는 시작점만 다를 뿐 정상 전개되고, ③은 AND 조건으로 특정 직급(인턴)을 가지치기해도 PRIOR가 있어 트리는 정상적으로 전개된다.

31 2과목

다음 [올림픽메달] 테이블에 대해 아래 SQL을 실행한 결과로 옳은 것은?

선수	국가	메달
김하나	KOR	골드
이두리	KOR	실버
박세찬	KOR	실버
정오름	USA	골드
최네오	USA	브론즈

```
SELECT 메달, COUNT(*) AS 수
FROM 올림픽메달
WHERE 국가 = 'KOR'
GROUP BY 메달;
```

① 골드 1, 실버 2, 브론즈 1

② 골드 1, 실버 2

③ 골드 1, 실버 1, 브론즈 1

④ 실버 2, 브론즈 1

정답 ②

핵심 해설

WHERE 국가='KOR' 로 먼저 행을 걸러내면 대상은 (김하나-골드, 이두리-실버, 박세찬-실버) 3건이다. 이를 메달로 GROUP BY 하면 '골드' 그룹 1건, '실버' 그룹 2건이 되어 결과는 골드 1, 실버 2 두 행이다. → 정답

선택지별 풀이

② .

① 브론즈는 KOR 데이터에 없으므로 나타날 수 없다.

③ 실버는 2건이므로 1이 아니다.

④ 브론즈가 KOR에 없어 오답. GROUP BY 는 WHERE 로 필터링된 행에 대해서만 그룹을 만들며, 그룹 함수 COUNT(*)는 각 그룹의 행 수를 센다는 점이 핵심이다.

32 2과목

이미 데이터가 저장되어 있는 [사원] 테이블의 현재 컬럼 정의가 아래와 같다. 각 ALTER 문을 실행할 때 오류가 발생하지 않는 것은? (Oracle 기준)

사원] 컬럼 정의

컬럼	데이터 타입	저장된 데이터
사번	NUMBER	사원 번호
이름	VARCHAR2(20)	15자 길이의 값 존재

컬럼	데이터 타입	저장된 데이터
급여	NUMBER(8,2)	정수부 6자리 값 존재

① `ALTER TABLE 사원 MODIFY 급여 NUMBER(4,2);`

② `ALTER TABLE 사원 MODIFY 이름 VARCHAR2(10);`

③ `ALTER TABLE 사원 MODIFY 사번 DATE;`

④ `ALTER TABLE 사원 MODIFY 급여 NUMBER(10,2);`

정답 ④

핵심 해설

컬럼에 데이터가 존재할 때 타입/크기 변경은 '기존 데이터를 손실 없이 담을 수 있는가'가 관건이다.

선택지별 풀이

④ NUMBER(8,2)→NUMBER(10,2) 는 정수부 자릿수를 6→8로 늘리는 크기 확대라 기존 데이터가 있어도 항상 허용된다. → 정답 ④.

① NUMBER(8,2)→NUMBER(4,2) 는 정수부를 2자리로 줄이는데 기존에 6자리 값이 있어 축소 불가 → 오류.

② VARCHAR2(20)→VARCHAR2(10) 은 15자 데이터가 이미 있어 담을 수 없어 오류.

③ 숫자(NUMBER)를 DATE 로 암시적 변환할 수 없어 오류. 일반적으로 크기 확대·호환 타입으로의 변경은 허용되지만, 축소·비호환 변경은 컬럼이 비어 있을 때만 가능하다.

---

33 2과목

다음 두 테이블에 대해 아래 SQL을 실행한 결과로 옳은 것은?

상품

상품ID
1
2
3

주문

상품ID
3

```
SELECT 상품ID
FROM 상품 A
WHERE NOT EXISTS (
SELECT 1 FROM 주문 B WHERE B.상품ID = A.상품ID
);
```

① 1 ② **1, 2** ③ (공집합) ④ 오류 발생

정답 ②

핵심 해설

NOT EXISTS 는 상관 서브쿼리가 한 건도 반환하지 않을 때 참이 되어 바깥 행을 남긴다. 상품ID 1·2 는 [주문]에 대응 행이 없어 서브쿼리가 공집합 → NOT EXISTS 참 → 결과에 포함. 상품ID 3 은 [주문]에 존재해 서브쿼리가 행을 반환 → NOT EXISTS 거짓 → 제외. 따라서 결과는 1, 2. → 정답

선택지별 풀이

② .

① 1만 나오는 경우는 없다(2도 대응 주문이 없음).

③ 공집합이 되려면 모든 상품이 주문에 있어야 한다.

④ 문법상 정상 실행된다. NOT IN 과 달리 NOT EXISTS 는 NULL 이 섞여도 예기치 않게 전체가 걸러지지 않는다는 점도 함께 기억해 둘 것.

34 2과목

다음 집합 연산자 중, 결과 집합에서 중복 행을 제거하지 않고 그대로 반환하는 것은?

① UNION
② UNION ALL
③ INTERSECT
④ EXCEPT (MINUS)

정답 ②

핵심 해설

UNION ALL 은 두 결과 집합을 단순히 이어 붙이며 중복 제거(정렬)를 수행하지 않아 중복 행이 그대로 남는다. → 정답

선택지별 풀이

② .

① UNION 은 합집합을 만들며 중복을 제거한다.
③ INTERSECT 는 교집합으로 중복을 제거한 공통 행만 반환한다.
④ EXCEPT(오라클 MINUS)는 차집합으로 역시 중복을 제거한다. UNION/INTERSECT/EXCEPT 는 내부적으로 정렬·중복 제거가 일어나 상대적으로 비용이 크지만, UNION ALL 은 그 과정이 없어 성능상 유리하다는 점도 함께 알아 둘 것.

35 2과목

다음 [이벤트] 테이블에 대해 아래 SQL을 실행할 때 반환되는 행의 개수는?

이벤트

일자
2024-01-15
2024-03-20
2024-05-10

```
SELECT EXTRACT(MONTH FROM 일자) FROM 이벤트
UNION
SELECT EXTRACT(YEAR FROM 일자) FROM 이벤트;
```

① 1개
② 2개
③ 3개
④ 4개

정답 ④

핵심 해설

EXTRACT(MONTH FROM 일자) 는 각 행에서 월을 뽑아 1, 3, 5 (서로 다른 3개)를 만든다. EXTRACT(YEAR FROM 일자) 는 세 행 모두 2024 이므로 2024 하나뿐이다. 두 결과를 UNION 하면 중복이 제거되어 {1, 3, 5, 2024} 4개 값이 남는다. → 정답

선택지별 풀이

④ 4개.

①
②
③ 은 각 SELECT 의 중복 제거·합집합 계산을 잘못한 경우다. UNION 은 자료형이 호환되는 두 결과를 하나의 컬럼으로 합치며 중복을 제거한다는 점, EXTRACT 는 날짜에서 특정 필드(연·월·일 등)를 정수로 추출한다는 점이 핵심이다.

36 2과목

다음 중 SQL 명령어에 대한 설명으로 옳지 않은 것은?

① TRUNCATE TABLE 은 WHERE 조건으로 특정 행만 지울 수 있는 DML 명령어이다.

② DELETE 문은 WHERE 절을 사용하여 테이블의 일부 행만 삭제할 수 있다.

③ TRUNCATE TABLE 은 테이블의 모든 행을 삭제하고 저장 공간을 초기화하며, 일반적으로 롤백할 수 없다.

④ DROP TABLE 은 테이블의 데이터뿐 아니라 구조(정의)까지 제거한다.

정답 ①

핵심 해설

TRUNCATE TABLE 은 DDL 명령어이며, 조건 없이 테이블의 모든 행을 한 번에 제거한다. WHERE 로 일부만 지울 수 없고 DML 도 아니다. 따라서

선택지별 풀이

① 은 두 가지 점(DML이라는 것, 조건 삭제 가능하다는 것)에서 틀린 설명이다. → 정답 ①.

② DELETE 는 DML 로 WHERE 로 일부 행만 지울 수 있다(옳음).

③ TRUNCATE 는 데이터를 모두 지우고 세그먼트를 초기 상태로 되돌리며 자동 커밋되어 롤백 불가(옳음).

④ DROP 은 데이터와 테이블 정의 자체를 삭제(옳음). DML(DELETE) 대비 DDL(TRUNCATE)은 개별 행 로그를 남기지 않아 대량 삭제 시 빠르다는 차이도 함께 기억할 것.

---

37 2과목

다음 두 테이블에 대해 아래 SQL을 실행한 결과로 옳은 것은?

테이블1

컬럼1	컬럼2
1	10
2	20
3	30
4	40

테이블2

컬럼1
1
2
3
4
5

```
SELECT SUM(A.컬럼2)
FROM 테이블1 A
INNER JOIN 테이블2 B ON A.컬럼1 = B.컬럼1;
```

① 120 ② NULL **③ 100** ④ 140

정답 ③

핵심 해설

INNER JOIN 은 양쪽 테이블에 조인 키가 모두 존재하는 행만 결합한다. 테이블1.컬럼1(1~4)과 테이블2.컬럼1(1~5)의 공통 값은 1,2,3,4 이며, 테이블2의 5 는 테이블1에 없어 매칭되지 않는다. 매칭된 4개 행의 테이블1.컬럼2 는 10+20+30+40 = 100 이다. → 정답

선택지별 풀이

③ 100.

① 120·④ 140 은 존재하지 않는 값(예: 50/60 등)을 잘못 더한 경우다.

② NULL 은 매칭 행이 하나도 없을 때만 SUM 이 NULL 이 되므로 여기서는 해당하지 않는다. 내부 조인에서는 한쪽에만 있는 키(값 5)가 결과에서 탈락한다는 점이 핵심이다.

38 2과목

다음 두 테이블을 조인 조건 없이 FROM 절에 함께 기술하여 조회할 때(카티션 곱), 반환되는 행의 개수는?

테이블X

(2개 행)
값
A
B

테이블Y

(2개 행)
값
1
2

```
SELECT *
FROM 테이블X, 테이블Y;
```

① 2개 ② 0개 **③ 4개** ④ 6개

정답 ③

핵심 해설

조인 조건이 없는 두 테이블을 FROM 절에 나열하면 카티션 곱(교차 조인)이 되어 두 테이블의 행 수를 곱한 만큼 결과가 생성된다. 테이블X 2행 × 테이블Y 2행 = 4행. → 정답

선택지별 풀이

③ 4개.

① 2개는 한쪽 행 수만 센 경우,

② 0개는 공통 조건이 없어 결과가 없다고 오해한 경우,

④ 6개는 2+... 등 잘못 계산한 경우다. 카티션 곱은 곱(2×2=4)이며, 조인 조건을 빠뜨리면 의도치 않게 대량의 행이 만들어질 수 있으니 주의해야 한다.

39 2과목

다음 MERGE 문을 실행한 후 [대상] 테이블의 상태로 옳은 것은?

대상

(실행 전)

ID	값
1	50
2	50

원본

ID	값
1	100
2	100

ID	값
3	100

```
MERGE INTO 대상 T
USING 원본 S
ON (T.ID = S.ID)
WHEN MATCHED THEN
UPDATE SET T.값 = S.값
WHEN NOT MATCHED THEN
INSERT (ID, 값) VALUES (S.ID, S.값);
```

① ID1=100, ID2=100, ID3=100 (3행)

② ID1=50, ID2=50, ID3=100 (3행)

③ ID1=100, ID2=100 (2행)

④ ID1=50, ID2=50 (2행)

정답 ①

핵심 해설

MERGE 는 원본 각 행을 대상과 ON 조건으로 비교한다. ID 1·2 는 대상에 이미 존재(MATCHED)하므로 UPDATE 로 값이 100 으로 갱신되고, ID 3 은 대상에 없으므로(NOT MATCHED) 값 100 으로 INSERT 된다. 결과 대상은 (1,100),(2,100),(3,100) 세 행 모두 100. → 정답

선택지별 풀이

① .

② 는 MATCHED 행이 갱신되지 않았다고 본 오답,

③ 은 NOT MATCHED INSERT 가 없었다고 본 오답,

④ 는 UPDATE·INSERT 둘 다 반영하지 않은 오답이다. MERGE 는 UPDATE(있으면 갱신)와 INSERT(없으면 삽입)를 한 문장으로 처리하는 UPSERT 구문이라는 점이 핵심이다.

---

40 2과목

다음 SQL 중 실행 시 오류가 발생하는 것은? (Oracle 기준, [판매] 테이블에는 제품명·단가·수량 컬럼이 있다)

① `SELECT 제품명, 단가 * 수량 AS 제품코드 FROM 판매 ORDER BY 제품코드;`

② `SELECT 제품명, 단가 FROM 판매 ORDER BY 2;`

③ `SELECT 제품명, SUM(수량) AS 총수량 FROM 판매 GROUP BY 제품명;`

④ `SELECT 제품명, 단가 * 수량 AS 제품코드 FROM 판매 WHERE 제품코드 > 1000;`

정답 ④

핵심 해설

SELECT 문의 논리적 처리 순서는 FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY 이다. 컬럼 별칭(AS 제품코드)은 SELECT 단계에서 비로소 정의되므로, 그보다 먼저 처리되는 WHERE 절에서는 별칭을 참조할 수 없다. 따라서

선택지별 풀이

④ 는 'WHERE 제품코드 > 1000'에서 별칭을 사용해 오류가 난다(원식 '단가*수량 > 1000'으로 써야 함). → 정답 ④.

① ORDER BY 는 SELECT 이후에 처리되어 별칭 참조가 허용되므로 정상.

② ORDER BY 2 는 SELECT 목록의 두 번째 컬럼(단가)으로 정렬하는 위치 지정 방식으로 정상.

③ GROUP BY 제품명 에 대해 SUM 집계로 정상 실행된다. 별칭은 WHERE·GROUP BY·HAVING 에서는 못 쓰고 ORDER BY 에서만 쓸 수 있다는 규칙이 핵심이다.

---

41 2과목

다음 중 연산 결과의 성격이 나머지 셋과 다른 하나는?

① `WHERE 1 > NULL`

② NULL + 1

③ NULL * 1　　④ NULL * NULL

정답 ①

핵심 해설

정답은 ①이다.

①의 1 > NULL은 값이 아니라 '비교 판정'으로, NULL과의 비교 결과는 항상 UNKNOWN이다. WHERE 절이 TRUE가 아닌 UNKNOWN이면 해당 행은 결과에서 제외되므로 어떤 행도 반환되지 않는다. 반면 ②·③·④는 산술식이며, 산술 연산에 NULL이 하나라도 포함되면 결과는 무조건 NULL 값이 된다(NULL+1=NULL, NULL*1=NULL, NULL*NULL=NULL). 즉 ②·③·④는 모두 'NULL이라는 값'을 산출하지만 ①은 '값'이 아니라 논리 3값(UNKNOWN) 판정으로 행 필터링에 작용한다는 점에서 다르다.

> **보충 개념** SQL의 논리값은 TRUE/FALSE/UNKNOWN 3값이며, NULL과의 등호·부등호 비교는 예외 없이 UNKNOWN이 된다. NULL 여부는 반드시 IS NULL / IS NOT NULL로 판정해야 한다.

42 2과목

다음 SELECT 문의 결과 값이 나머지 셋과 다른 하나는?

① `SELECT 1 + NULL`　　② `SELECT 1 * NULL`

③ `SELECT 'X'`　　④ `SELECT NULL, NULL`

정답 ③

핵심 해설

정답은 ③이다.

③은 상수 문자값 'X'를 반환하므로 결과가 NULL이 아닌 명확한 값이다. 나머지는 모두 NULL과 관련된 결과를 낸다:

선택지별 풀이

① 1 + NULL은 산술식에 NULL이 포함되어 NULL,

② 1 * NULL도 같은 이유로 NULL,

④ SELECT NULL, NULL은 두 개의 NULL 컬럼을 반환한다. 따라서 값이 존재하는(non-null) 유일한 항목은 ③이다.

> **보충 개념** NULL은 '값이 없음(미지의 상태)'을 뜻하므로 어떤 산술 연산과 결합해도 결과가 NULL로 전파된다. 값을 강제로 채우려면 NVL/COALESCE 같은 함수를 사용한다.

43 2과목

다음 SQL을 순서대로 실행했을 때, 마지막 SELECT의 결과 값으로 옳은 것은?

```
CREATE TABLE 상품 (
상품번호 INT IDENTITY(1,1),
수량   INT CHECK (수량 > 0)
);
INSERT INTO 상품 (수량) VALUES (-1);
INSERT INTO 상품 (수량) VALUES (0);
INSERT INTO 상품 (수량) VALUES (1);
SELECT COUNT(*) FROM 상품;
```

① 0　　② **1**　　③ 2　　④ 3

정답 ②

핵심 해설

정답은 ②(1)이다. 수량 컬럼에는 CHECK (수량 > 0) 제약이 걸려 있으므로 0보다 큰 값만 삽입이 허용된다. -1은 조건 위반으로 거부, 0도 '초과(>)'가 아니라 거부되며, 1만 조건(1 > 0)을 만족해 정상 삽입된다. 따라서 실제로 저장된 행은 1건이고

COUNT(*)는 1이 된다. IDENTITY(1,1)은 시드 1, 증가치 1로 자동 채번될 뿐 행 수에는 영향을 주지 않는다(삽입 실패 행은 카운트되지 않음).

> **보충 개념** CHECK 제약은 컬럼 값이 지정한 논리식을 만족할 때만 DML을 허용하며, 경계값(=0)은 부등호에 포함되지 않으므로 거부된다는 점에 유의한다.

44 2과목

뷰 사원뷰가 테이블 사원을 참조하고 있다. 이 상태에서 다음 문장을 실행할 때 실제로 삭제되는 대상으로 옳은 것은?

```
DROP TABLE 사원 RESTRICT;
```

① 아무 것도 삭제되지 않는다
② 사원 테이블만 삭제된다
③ 사원 테이블과 사원뷰가 함께 삭제된다
④ 사원뷰만 삭제된다

정답 ①

핵심 해설

정답은 ①이다. RESTRICT 옵션은 삭제하려는 객체를 참조(의존)하는 다른 객체가 하나라도 존재하면 삭제를 거부한다. 사원뷰가 사원 테이블에 의존하고 있으므로 DROP은 오류로 실패하고, 결과적으로 테이블도 뷰도 아무 것도 삭제되지 않는다. ②·③은 참조 관계를 무시한 삭제로 RESTRICT 의미에 어긋난다(참조 객체를 함께 지우는 것은 CASCADE의 동작이다).
④ 뷰만 삭제되는 일은 없다.

> **보충 개념** 반대로 DROP TABLE 사원 CASCADE를 쓰면 의존 객체(뷰 등)까지 연쇄 삭제된다. RESTRICT는 안전하게 '의존 있으면 거부', CASCADE는 '의존까지 제거'로 대비된다.

45 2과목

함수 NVL(수당, 급여)에 대한 설명으로 옳지 않은 것은?

① 수당이 NULL이면 급여를 반환한다
② 수당이 NULL이 아니면 수당 값을 그대로 반환한다
③ 첫 번째 인자와 두 번째 인자의 데이터 타입은 서로 호환되어야 한다
④ 수당과 급여를 더한 합계를 반환한다

정답 ④

핵심 해설

정답(옳지 않은 것)은 ④이다. NVL(expr1, expr2)는 첫 번째 인자 expr1이 NULL이면 expr2를, NULL이 아니면 expr1을 반환하는 'NULL 치환' 함수일 뿐, 두 인자를 더하거나 산술 연산을 수행하지 않는다.

선택지별 풀이

① 수당이 NULL이면 대체값인 급여를 반환하므로 옳다.
② 수당이 NULL이 아니면 수당을 그대로 반환하므로 옳다.
③ 두 인자의 데이터 타입은 호환(암시적 변환 가능)되어야 하므로 옳다.

> **보충 개념** 세 개 이상 인자의 우선순위 치환은 COALESCE, 조건 분기 치환은 NVL2(expr, v1, v2)를 사용한다.

46 2과목

급여가 높은 '상위 3개 급여 수준'을 뽑되, 동일 급여자는 같은 순위로 모두 포함하고 순위를 건너뛰지 않아야 한다. 다음 빈칸에 들어갈 함수로 가장 적절한 것은?

```
SELECT 이름, 급여
FROM (
SELECT 이름, 급여,
```

```
( ) OVER (ORDER BY 급여 DESC) AS 순위
FROM 사원
)
WHERE 순위 <= 3;
```

① RANK() ② **DENSE_RANK()** ③ ROW_NUMBER() ④ NTILE(3)

정답 ②

핵심 해설

정답은 ②(DENSE_RANK)이다. DENSE_RANK는 동일 값에 같은 순위를 부여하면서 다음 순위를 건너뛰지 않으므로 (1,1,2,3…), 순위 ≤ 3 조건이 '서로 다른 상위 3개 급여 수준'과 그 수준에 속한 모든 동순위자를 정확히 반환한다.

선택지별 풀이

① RANK는 동순위 뒤 순위를 건너뛰어(1,1,3…) 3개 수준을 다 못 뽑을 수 있다.
③ ROW_NUMBER는 동순위여도 임의로 1,2,3을 매겨 같은 급여인데 일부만 나오는 문제가 생긴다.
④ NTILE(3)은 전체 행을 3개 그룹으로 등분하는 함수로 '상위 3개 급여 수준'과 무관하다.

> **보충 개념** '상위 N명(정확히 N행)'이면 ROW_NUMBER, '상위 N위(동순위 포함, 결번 허용)'면 RANK, '상위 N수준(동순위 포함, 결번 없음)'이면 DENSE_RANK를 선택한다.

47 2과목

부서 테이블에 존재하지 않는 부서번호를 가지거나 부서번호가 NULL인, 즉 '어느 부서에도 속하지 않는' 사원을 모두 조회하려 한다. 가장 적절한 SQL은?

① `SELECT * FROM 사원 WHERE 부서번호 = NULL`

② `SELECT s.* FROM 사원 s INNER JOIN 부서 d ON s.부서번호 = d.부서번호 WHERE d.부서번호 IS NULL`

③ `SELECT * FROM 사원 WHERE 부서번호 IN (SELECT 부서번호 FROM 부서)`

④ `SELECT s.* FROM 사원 s LEFT OUTER JOIN 부서 d ON s.부서번호 = d.부서번호 WHERE d.부서번호 IS NULL`

정답 ④

핵심 해설

정답은 ④이다. LEFT OUTER JOIN으로 모든 사원을 보존한 뒤, 조인에 실패해 부서 쪽 컬럼이 NULL로 채워진(=매칭되는 부서가 없는) 행만 d.부서번호 IS NULL로 걸러낸다. 이 방식은 '부서번호가 부서 테이블에 없는 고아 값'과 '부서번호 자체가 NULL인 사원'을 모두 포착한다.
① = NULL은 항상 UNKNOWN이라 결과가 비어 잘못된 표현이다(반드시 IS NULL 사용).
② INNER JOIN은 매칭된 행만 남기므로 d.부서번호가 NULL일 수 없어 항상 공집합이다.
③은 오히려 '부서에 속하는' 사원을 반환하며(반대 결과), 게다가 부서번호가 NULL인 사원은 IN 비교에서 제외된다.

> **보충 개념** 안티조인은 OUTER JOIN + IS NULL 또는 NOT EXISTS로 구현하며, NOT IN은 서브쿼리 결과에 NULL이 섞이면 전체가 UNKNOWN이 되어 위험하다.

48 2과목

`GROUP BY ROLLUP(부서, 직급)`을 수행할 때 생성되는 집계(그룹핑) 조합을 (부서, 직급) 형태로 나타냈다. 이 중 ROLLUP의 결과에 포함되지 않는 조합은? (여기서 a=특정 부서, b=특정 직급, NULL=소계 표시)

① (a, b) ② (a, NULL) ③ **(NULL, b)** ④ (NULL, NULL)

정답 ③

핵심 해설

정답은 ③이다. ROLLUP(부서, 직급)은 지정한 컬럼의 '계층적 부분합'만 만든다. 즉 (부서, 직급) 상세 → (부서) 소계 → 전체합계 순으로 오른쪽 컬럼부터 차례로 제거한 조합만 생성하므로 그 결과는 (a,b), (a,NULL), (NULL,NULL) 세 종류다.

③ (NULL, b)처럼 '앞 컬럼(부서)만 없애고 뒤 컬럼(직급)만 남긴' 조합은 ROLLUP에서는 만들어지지 않는다. 이 (NULL, b) 조합까지 모두 얻으려면 대칭적으로 모든 부분집합을 생성하는 CUBE(부서, 직급)를 사용해야 한다.

> **보충 개념** ROLLUP은 n+1개, CUBE는 2ⁿ개의 그룹핑 집합을 만든다. 특정 조합만 필요하면 GROUPING SETS로 명시한다.

49 2과목

특정 컬럼에 입력되는 값이 지정한 조건식(범위·목록 등)을 만족하도록 강제하여, 조건을 위반하는 값의 삽입·수정을 거부하는 제약조건은?

① NOT NULL ② **CHECK** ③ PRIMARY KEY ④ FOREIGN KEY

정답 ②

핵심 해설

정답은 ②(CHECK)이다. CHECK 제약은 예를 들어 CHECK (수량 > 0), CHECK (성별 IN ('M','F'))처럼 컬럼 값이 지정 논리식을 만족할 때만 DML을 허용하고, 위반하는 값은 거부한다.

선택지별 풀이

① NOT NULL은 'NULL을 허용하지 않음'만 강제할 뿐 값의 범위/목록은 검사하지 않는다.
③ PRIMARY KEY는 유일성 + NOT NULL로 행을 식별하는 제약이다.
④ FOREIGN KEY는 참조 무결성(다른 테이블의 키 값 존재)을 보장하는 제약으로 조건식 검사와는 다르다.

> **보충 개념** 값의 도메인(허용 범위/집합)을 제한하는 것이 CHECK의 역할이며, DEFAULT는 미입력 시 기본값 지정으로 검사가 아니다.

50 2과목

다음 SQL의 결과가 의미하는 대상으로 옳은 것은?

```
SELECT 고객번호
FROM 주문
GROUP BY 고객번호
HAVING MAX(수량) > 10;
```

① 주문 수량의 총합이 10을 초과한 고객
② 주문 횟수가 10회를 초과한 고객
③ 모든 주문의 수량이 10을 초과한 고객
④ **주문 수량이 10을 초과한 주문을 한 번이라도 한 고객**

정답 ④

핵심 해설

정답은 ④이다. 고객번호로 그룹화한 뒤 HAVING MAX(수량) > 10은 '그 고객의 주문 중 최대 수량이 10을 초과'하는 그룹만 남긴다. 어떤 그룹의 최댓값이 10보다 크다는 것은 그 고객이 수량 10을 넘는 주문을 적어도 한 건 했다는 뜻이므로

선택지별 풀이

④ 가 옳다.
① 총합 조건이라면 SUM(수량) > 10이어야 한다.
② 주문 횟수라면 COUNT(*) > 10이어야 한다.
③ '모든 주문이 10 초과'는 최솟값 조건 MIN(수량) > 10이어야 하며 MAX와 다르다.

> **보충 개념** HAVING은 GROUP BY로 만들어진 그룹에 대한 필터로, 집계함수(MAX/MIN/SUM/COUNT/AVG) 결과를 조건으로 사용한다. 개별 행 필터인 WHERE와 구분해야 한다.